

Day 1: Introduction to Angular

SECTIONS:-

- 1) Internship Objective
- 2) Frontend, Backend, and Database Overview
- 3) Software to Install
- 4) To create and run the server
- 5) Structure of Src folder
- 6) Component

1. INTERNSHIP OBJECTIVE:-

The purpose of this internship is to train students to become Full-Stack Web Developers. By the end of the internship, students should be able to create a complete web application based on a client's requirements.

For example, if a shopkeeper asks for a web application to manage a store, students should be able to develop it. Similarly, if a client requests a hospital management system, students should be able to create that as well. The main objective is to build the confidence and skills required to develop web applications for different types of clients and business needs.

In a way, web development is similar to a skilled trade. A client approaches you with a requirement, and your job is to build a solution that meets that requirement. To do this effectively, you must become proficient in the necessary technologies and development practices. Consider the example of a carpenter. A carpenter may build tables, chairs, cabinets, and many other

types of furniture based on a customer's requirements. Although each project may be different, the carpenter relies on the same fundamental skills, tools, and techniques to complete the work. Similarly, a web developer may be asked to create different types of applications, but the underlying development skills remain the same. As experience grows, developers become more confident in handling new and unfamiliar requirements.

2. Frontend, Backend and Database Overview

A complete web application can be divided into three major domains: Frontend, Backend, and Database. A developer who works across all three domains is known as a Full-Stack Developer.

In this internship, the following technologies will be used:

Frontend (Client-Side):

- HTML
- CSS
- Angular
- Editor: Visual Studio Code (VS Code)

Backend (Middleware/ Server-Side):

- Java
- Spring Boot
- Editor: Spring Tool Suite (STS) or Eclipse

Database (Server-Side/ Data Layer):

- MySQL

- Oracle
- Tools: MySQL Workbench and SQL+ (for Oracle)

Example Story-

Suppose you come to Silicon with some admission-related queries. You cannot directly go to the Admin Office with your questions. The rule is that you must first meet the security guard and provide your registration number.

The security guard has a box containing several chits. He writes your registration number on a chit and takes the box to the Admin Office. The security guard asks the Admin Officer to verify whether the students who have come actually have valid admissions.

The Admin Officer does not check the records himself. Instead, he forwards the chits to the Register Guy. The Register Guy is responsible for maintaining all the admission registers stored in a cupboard.

The Register Guy checks whether each registration number is present in the registers. After verification, he marks the result and sends the chits back to the Admin Officer. The Admin Officer then forwards the chits to the security guard.

The security guard returns outside and allows only those students whose chits have been marked as valid. Those students are now permitted to enter the Admin Office and ask their queries.

In this story:

- The students represent the requests.
- The security guard represents the Frontend Layer.

- The Admin Officer represents the Middleware/Backend Layer.
- The Register Guy and his registers represent the Database Layer.

3) Software to Install:-

1. Node JS (LTS)
2. Angular 15
3. VSCode

4) To Create and run the server:-

Angular creates a template application for us. Think of it as a blank skeleton on which we will build our application.

To create a new Angular project, open the Command Prompt and run the following command:

```
ng new Project_Name
```

Angular will create a project folder containing several files and folders required for the application.

To run the project, navigate to the project directory using the `cd` command and execute:

```
ng serve
```

This command starts the Angular development server. Once the server starts successfully, Angular will provide a host address and port number, such as:

`http://localhost:4200`

Open a web browser and enter this address. You should see the default Angular welcome page, which confirms that the application is running successfully.

(Note: In these notes, the terms "project" and "server" may sometimes be used interchangeably when referring to the running Angular application.)

5) Structure of Src folder:-

The `src` folder is where we will write most of our code. During the initial stages of learning Angular, we can ignore most of the files and folders outside this directory.

The `src` folder contains two folders and several files. The two folders are:

- `app`
- `assets`

The files are:

- `index.html`
- `main.ts`
- `styles.css`

- `favicon.ico`

The first file that runs in the background when the application starts is `main.ts`. Its job is to bootstrap `app.module.ts`. For now, think of "bootstrap" as simply meaning "load".

The first page rendered in the browser is `index.html`. This file is connected to `app.component.html`.

The `styles.css` file is used for global styles that apply to the entire application. The `favicon.ico` file is the small icon displayed in the browser tab and bookmarks section. For now, these two files can be ignored.

Our main work will be inside the App Component.

Connection Flow:

`main.ts` → `app.module.ts` → `app.component.ts`

`index.html` → `app.component.html`

The `assets` folder is used to store static resources such as images, videos, icons, and other files that may be used by the application.

The `app` folder is the main folder where we will do most of our development work.

The `app` folder is both a Module and a Component.

Trick:

- If a folder contains a `module.ts` file, it is usually a Module.
- If a folder contains `component.ts`, `component.html`, `component.css`, and `component.spec.ts`, it is usually a Component.

You can verify this yourself by creating a new module and a new component.

(Note: These observations are based on Angular 15. Other Angular versions may have slightly different file names)

The `app` folder contains the following files:

1) `app.module.ts`

Since `app` is also a module, this file acts as a registry file.

In this file, you declare:

- What you are importing from other modules.
- What you are exporting to other modules.
- Which components belong to this module.

Think of it as a central registration file that keeps track of the module's dependencies and declarations.

2) `app.component.ts`

Since `app` is also a component, this file contains the main `AppComponent` class.

Every Angular component has a class.

Trick:

(The class name is usually written in PascalCase and can be found directly inside the `.ts` file.)

Inside this class, you will write the component's logic, similar to how you write logic inside Java classes.

A component class can contain:

- Attributes (variables)
- Methods (functions)
- Constructors
- Other TypeScript logic

We will explore these concepts further when creating our own components.

3)app-routing.module.ts

- This is another registry file. It is used to register routes in the Router Module.
- We will need to create something called "routes" in the future. In this file, you import the Router Module classes. Then, you create a routes object and fill it with the routes used by your application.
- After filling the routes object, you pass it to the Router Module and export it so that other modules can import and use it.
- In simple terms, this file is responsible for storing and registering all the routes used by the application.
-

4) app.component.html

This file contains the HTML template of the component.

Whatever HTML is written here is displayed on the screen when the component is rendered.

Think of this file as the visual part of the component.

Examples:

- Headings
- Paragraphs
- Buttons
- Forms
- Tables
- Images

All user-visible content is generally written here.

5) app.component.css

This file contains the styles for the component.

The CSS written here applies only to this component by default.

Think of it as the component's personal styling file.

Examples:

- Text color
- Background color
- Margins
- Padding
- Font size
- Layout styling

Unlike `styles.css`, which affects the whole application, `app.component.css` mainly affects only the App Component.

6) app.component.spec.ts

This file is used for testing.

Angular automatically creates it when a component is generated.

The purpose of this file is to verify that the component behaves correctly.

Examples:

- Checking whether the component is created successfully.
- Checking whether methods return expected values.
- Checking whether certain elements appear on the page.

For beginners, this file can usually be ignored until testing is introduced later.

6) Component :-

A component is somewhat similar to the concept of objects in OOP or entities in a database. Basically, a component is a "thing" in your application.

For example, an entire page can be a component. A UI element such as a header, footer, navbar, or sidebar can also be a component. What should be a component and what should not be a component is ultimately decided by the developer.

With experience, developers develop a sense of how to break an application into components. Similar to how practicing many ER diagrams helps you identify entities in a new problem, building many Angular applications helps you identify components in a new project.

Any further understanding of components can only be achieved by actually creating and using them.

A component can be created using the following command:

```
ng g c component-name
```

(It is preferable to use kebab-case while naming components. Angular will automatically generate the corresponding class name in PascalCase, which makes importing and exporting components easier.)

A component consists of three main files (for now, you can ignore the `.spec.ts` file since it is mainly used for testing):

- An HTML file – for the template.
- A CSS file – for the styling.
- A TypeScript file – for the logic.

Note:

(If you run `ng g c component-name` from the project's root directory, Angular will create the component inside the `app` folder by default.

If you want to create nested components (as required in Task 1), first navigate into the target component's folder using the `cd` command and then create the new component from there.)

Every component has something called a selector. A selector is simply the name used to place one component inside another component. You can find the selector inside the `@Component` decorator in the `.ts` file. To use a component, copy its selector and use it as an HTML tag inside another component's HTML file. This is how Angular connects and displays components within other components.

THANK YOU.